

Guía práctica 03 - 01

Paso 0: Requisitos previos

Antes de comenzar, asegúrate de tener estas bibliotecas instaladas:

```
pip install opencv-python cvlib tensorflow
```

Tensorflow actualmente funciona con la versión 3.10 de Python, en caso no tengas esta versión instala de esta manera:

Paso 01: Ejecuta el instalador. MUY IMPORTANTE:

- Marca la opción **"Add Python 3.10 to PATH"**
- Luego haz clic en **"Customize installation"**
- Asegúrate de que estén marcadas las opciones:
 - pip
 - tcl/tk and IDLE
 - Add Python to environment variables

Luego haz clic en "Next" > "Install".

Paso 2: Crear un entorno virtual con Python 3.10

Una vez instalado Python 3.10:

Abre una nueva terminal (CMD) o PowerShell

Escribe lo siguiente para verificar que se instaló bien:

```
py -3.10 --version
```

Debe aparecer algo como:

```
Python 3.10.0
```

Crea un entorno virtual llamado tf-env: `py -3.10 -m venv tf-env`

Esto creará una carpeta tf-env en el directorio actual.

Paso 3: Activar el entorno virtual

Una vez creado el entorno, actívalo así:

```
.\tf-env\Scripts\activate
```

Después de esto, deberías ver algo como:

```
(tf-env) C:\Users\TuUsuario>
```

Paso 4: Instalar TensorFlow

Con el entorno activado, ejecuta:

```
pip install --upgrade pip
```

pip install tensorflow

Luego de asegurarse de tener instalado todo lo necesario continuar:

Paso 1: Importar las librerías necesarias

```
import cv2
```

```
import cvlib as cv
```

```
from cvlib.object_detection import draw_bbox
```

Explicación:

- cv2: permite trabajar con video y procesamiento de imágenes.
- cvlib: es una librería que facilita la detección de objetos usando modelos preentrenados.
- draw_bbox: dibuja las cajas con nombre y porcentaje de confianza sobre cada objeto detectado.

Paso 2: Encender la cámara

```
webcam = cv2.VideoCapture(0)
```

Explicación:

- Esto abre la cámara por defecto de la computadora.
- El 0 representa la cámara principal (si tienes varias, puedes usar 1, 2, etc.).

Paso 3: Procesar cada cuadro (frame) en tiempo real

```
while True:
```

```
    status, frame = webcam.read()
```

```
    if not status:
```

```
        break
```

Explicación:

- webcam.read() obtiene una imagen (frame) de la cámara.
- status indica si la lectura fue exitosa.
- El while True mantiene el video encendido hasta que lo apagues.

Paso 4: Detectar objetos

```
bbox, label, conf = cv.detect_common_objects(frame)
```

Explicación:

- detect_common_objects() analiza la imagen y devuelve:

- bbox: coordenadas de las cajas alrededor de cada objeto.
- label: nombre del objeto (por ejemplo, 'person', 'cell phone').
- conf: nivel de confianza (qué tan seguro está el modelo de su predicción).

Paso 5: Dibujar las cajas con etiquetas

```
output = draw_bbox(frame, bbox, label, conf)
```

Explicación:

- Esta función dibuja un rectángulo y escribe el nombre y el porcentaje de confianza sobre cada objeto.

Paso 6: Mostrar el video con los objetos detectados

```
cv2.imshow("Object Detection", output)
```

Explicación:

- Se abre una ventana llamada "Object Detection" que muestra el resultado en tiempo real.

Paso 7: Salir del programa con una tecla

```
if cv2.waitKey(1) == 27: # ESC para salir
```

```
    break
```

Explicación:

- waitKey(1) espera una milésima de segundo por una tecla.
- Si se presiona **ESC (código 27)**, se rompe el bucle y el programa termina.

Paso 8: Liberar recursos

```
webcam.release()
```

```
cv2.destroyAllWindows()
```

Explicación:

- Libera la cámara.
- Cierra todas las ventanas de OpenCV.

Actividades sugeridas:

1. **Identificación de objetos:**

- ¿Qué objetos pueden reconocer? (personas, laptop, celular, silla, etc.) Captura pantalla y agrega tus evidencias.

2. **Actividad:**

- Haz que la cámara detecte una persona con un celular. ¿Aparecen los dos objetos al mismo tiempo?

3. **Personalización:**

- Cambiar el color de las cajas o la fuente del texto. Agrega evidencias.
- Mostrar solo objetos con más del 70% de confianza. Agrega evidencias